

2. O atacante pode interceptar a resposta de um servidor DNS e alterá-la ou criar uma nova resposta totalmente falsa com o intuito de direcionar o usuário para um *site* ou domínio de sua escolha. Pode-se prevenir esse tipo de ataque por meio da autenticação da origem da mensagem e verificação de sua integridade (ver Capítulo 10).
3. O atacante pode inundar o servidor DNS com requisições com o objetivo de sobrecarregá-lo ou até derrubá-lo. Esse tipo de ataque pode ser evitado usando algum mecanismo contra ataques de negação de serviço.

Para proteger o DNS, a IETF desenvolveu uma tecnologia chamada *DNS Security* (DNSSEC – Segurança do DNS), que fornece autenticação da origem e integridade da mensagem usando um serviço de segurança denominado *assinatura digital* (ver Capítulo 10). O DNSSEC, no entanto, não fornece confidencialidade para as mensagens DNS. Não há proteção específica contra ataques de negação de serviço na especificação do DNSSEC, mas, o sistema de *cache*, até certo ponto, protege os servidores de nível mais alto contra estes ataques.

2.4 PARADIGMA *PEER-TO-PEER*

Discutimos o paradigma cliente-servidor no início do capítulo. Discutimos também algumas aplicações cliente-servidor padronizadas nas seções anteriores. Nesta seção, discutimos o paradigma *peer-to-peer* (também conhecido como *par a par*). O primeiro exemplo de compartilhamento de arquivos via *peer-to-peer* remonta a dezembro de 1987, quando Wayne Sino criou o *WWIVnet*, o componente de rede do Guerra Mundial Quatro (WWIV – World War Four), um Sistema de Quadro de Avisos (também conhecido como *Bulletin Board System*). Em julho de 1999, Ian Clarke projetou a Freenet, um sistema de armazenamento de dados descentralizado, distribuído e resistente a censuras, com o objetivo de fornecer liberdade de expressão através de uma rede *peer-to-peer* com forte proteção de anonimato.

O paradigma *peer-to-peer* ganhou popularidade com o Napster (1999-2001), um serviço de compartilhamento de arquivos de música *online* criado por Shawn Fanning. Embora a cópia e distribuição livre de arquivos de música pelos usuários tenham levado a uma ação judicial contra o Napster por violação de direitos autorais, o que causou o fechamento do serviço, ele abriu o caminho para modelos de distribuição de arquivos baseados em *peer-to-peer* que surgiram mais tarde. O Gnutella foi lançado em março de 2000, seguido pelo Fast-Track (usado pelo Kazaa), BitTorrent, WinMX e GUNet em março, abril, maio e novembro de 2001, respectivamente.

2.4.1 Redes P2P

Usuários da Internet que queiram compartilhar os seus recursos tornam-se *peers* e formam uma rede. Quando um *peer* na rede tem um arquivo (por exemplo, um arquivo de áudio ou de vídeo) para compartilhar, ele o deixa disponível para o restante dos *peers*. Um *peer* interessado pode se conectar ao computador onde o arquivo está armazenado e fazer o *download*. Depois de um *peer* obter um arquivo, ele pode deixá-lo disponível para outros *peers* para *download*. À medida que os *peers* se juntam à rede e obtêm o arquivo, mais cópias passam a ficar disponíveis para o grupo. Como as listas de *peers* podem crescer ou diminuir, a questão é como o paradigma rastreia os *peers* legais e a localização dos arquivos. Para responder a estas perguntas, precisamos primeiro dividir as redes P2P em duas categorias: centralizadas e descentralizadas.

Redes centralizadas

Em uma rede P2P centralizada, o sistema de diretórios – a lista dos *peers* e do que eles têm a oferecer – usa o paradigma cliente-servidor, mas o armazenamento e o *download* dos arquivos são feitos

usando o paradigma *peer-to-peer*. Assim, uma rede P2P centralizada é muitas vezes denominada rede P2P híbrida. O Napster foi um exemplo de uma rede P2P centralizada. Nesse tipo de rede, um *peer* primeiro se registra junto a um servidor central. Em seguida, o *peer* fornece o seu endereço IP e uma lista de arquivos que ele tem para compartilhar. Para evitar o colapso do sistema, o Napster usava vários servidores para isso, apesar de mostrarmos apenas um na Figura 2.46.

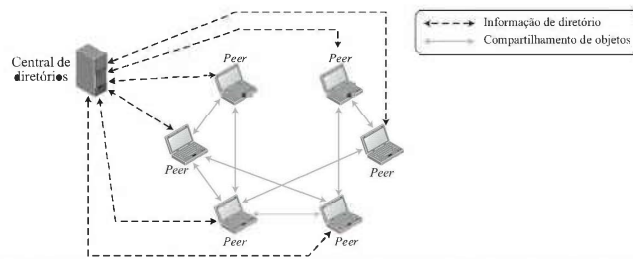


Figura 2.46 Rede centralizada.

Um *peer*, ao procurar um determinado arquivo, envia uma consulta para um servidor central. O servidor procura em seu diretório e responde com os endereços IP dos nós que têm uma cópia do arquivo. O *peer* entra em contato com um dos nós e faz o *download* do arquivo. O diretório é constantemente atualizado à medida que os nós entram ou saem da rede.

Redes centralizadas simplificam a manutenção dos diretórios, mas apresentam vários inconvenientes. O acesso ao diretório pode gerar uma quantidade enorme de tráfego e deixar o sistema lento. Os servidores centrais são vulneráveis a ataques, e se todos eles falharem, o sistema inteiro deixa de funcionar. O componente central do sistema foi o principal responsável pelo Napster perder a ação judicial por violação de direitos autorais e pelo seu fechamento em julho de 2001. A Roxio trouxe de volta o Novo Napster em 2003; o Napster versão 2 é agora um *site* legal para venda de músicas.

Rede descentralizada

Uma rede P2P descentralizada não depende de um sistema de diretórios centralizado. Nesse modelo, os *peers* se organizam em uma rede sobreposta (*overlay network*), uma rede lógica criada sobre uma rede física. Dependendo de como os nós da rede sobreposta são ligados, uma rede P2P descentralizada pode ser classificada como não estruturada ou estruturada.

Redes não estruturadas

Em uma rede P2P não estruturada, os nós se ligam de forma aleatória. Uma busca em uma rede P2P não estruturada não é muito eficiente, pois uma consulta para localizar um arquivo deve inundar toda a rede (ou seja, ser enviada a todos os nós), o que produz uma quantidade significativa de tráfego e pode não retornar resultados. Dois exemplos desse tipo de rede são Gnutella e Freenet. A seguir, discutimos a rede Gnutella como um exemplo.

Gnutella A rede Gnutella é um exemplo de uma rede *peer-to-peer* descentralizada, porém não estruturada, no sentido de que o diretório é aleatoriamente distribuído entre os nós. Quando o nó A quer acessar um objeto (tal como um arquivo), ele entra em contato com um de seus vizinhos.

Um vizinho, nesse caso, é qualquer nó cujo endereço seja conhecido pelo nó A. O nó A envia uma mensagem de *consulta* ao seu vizinho, o nó W. A consulta inclui o identificador do objeto (por exemplo, o nome do arquivo). Se o nó W sabe o endereço do nó X, que tem o objeto, ele envia uma mensagem de *resposta*, a qual inclui o endereço do nó X. O nó A agora pode usar os comandos definidos em um protocolo de transferência, como HTTP, para obter do nó X uma cópia do objeto. Se o nó W não sabe o endereço do nó X, ele *inunda* o pedido de A para todos os seus vizinhos. Por fim, um dos nós da rede responde à mensagem de *consulta*, e o nó A ganha acesso ao nó X. Discutiremos o mecanismo de inundação no Capítulo 4, quando discutimos protocolos de roteamento, mas é importante mencionar aqui que, embora as inundações no Gnutella sejam de certa forma controladas para evitar elevadas cargas de tráfego, uma das razões pelas quais o Gnutella apresenta baixa escalabilidade é o mecanismo de inundação.

Uma das perguntas que ainda precisam ser respondidas é como, de acordo com o processo descrito anteriormente, o nó A sabe o endereço de pelo menos um vizinho. Isto é feito no momento da inicialização (*bootstrap*), quando o nó instala o *software* Gnutella pela primeira vez. O *software* inclui uma lista de nós (*peers*) que o nó A pode registrar como vizinhos. Mais tarde, o nó A pode usar duas mensagens, chamadas *ping* e *pong*, para tentar descobrir se um vizinho ainda está ativo.

Conforme mencionado anteriormente, um dos problemas com a rede Gnutella é a falta de escalabilidade causada pelo mecanismo de inundação. Quando o número de nós aumenta, as inundações não levam a um bom resultado. Para tornar a consulta mais eficiente, uma nova versão do Gnutella implementou um sistema hierárquico de *folhas* e *ultranós*. Um nó entrante na rede torna-se uma folha, não se responsabilizando pelo roteamento; nós que são capazes de auxiliar no roteamento são promovidos a *ultranós*. Isto permite que as consultas propaguem-se até mais longe, aumentando a eficiência e a escalabilidade do sistema. O Gnutella adotou uma série de outras técnicas, como a inclusão do Protocolo de Roteamento de Consulta (QRP – Query Routing Protocol) e da Consulta Dinâmica (DQ – Dynamic Querying) para reduzir a carga de tráfego e tornar as pesquisas mais eficientes.

Redes estruturadas

Uma rede estruturada utiliza um conjunto predefinido de regras para ligar nós para que as consultas sejam efetiva e eficientemente resolvidas. A técnica mais comum utilizada com essa finalidade é a Tabela de *Hash* Distribuída (DHT – Distributed Hash Table). A DHT é usada em muitas aplicações, incluindo Estrutura de Dados Distribuída (DDS – Distributed Data Structure), Sistemas de Conteúdos Distribuídos (CDS – Content Distributed Systems), Sistema de Nomes de Domínio (DNS – Domain Name System) e compartilhamento de arquivos P2P. Um protocolo de compartilhamento de arquivos P2P popular que usa DHT é o BitTorrent. Discutiremos o conceito de DHT independentemente na próxima seção como uma técnica que pode ser utilizada tanto em redes P2P estruturadas como em outros sistemas.

2.4.2 Tabela de *Hash* Distribuída

Uma Tabela de *Hash* Distribuída (DHT – Distributed Hash Table) distribui dados (ou referências a dados) entre um conjunto de nós de acordo com algumas regras predefinidas. Cada *peer* em uma rede baseada em DHT torna-se responsável por uma série de dados. Para evitar a sobrecarga causada pelas inundações que discutimos no caso das redes P2P não estruturadas, redes baseadas em DHT permitem que cada *peer* tenha conhecimento parcial sobre a rede toda. Este conhecimento pode ser utilizado para rotear as consultas por itens até os nós responsáveis por eles, usando procedimentos eficazes e escaláveis que iremos discutir em breve.

Espaço de endereços

Em uma rede baseada em DHT, cada *peer* e cada conjunto de dados é mapeado para um ponto em um grande espaço de endereços de tamanho 2^n . O espaço de endereços é concebido usando

aritmética modular, o que significa que podemos pensar nos pontos do espaço de endereços como se eles fossem distribuídos uniformemente sobre um círculo com 2^m pontos (0 a $2^m - 1$), em sentido horário, conforme apresentado na Figura 2.47. A maioria das implementações de DHT usa $m = 160$.

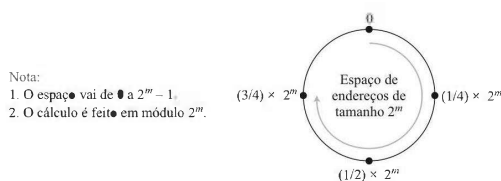


Figura 2.47 Espaço de endereços.

Calculando o *hash* do identificador do *peer*

O primeiro passo na criação do sistema DHT é colocar todos os *peers* sobre o anel que representa o espaço de endereços. Para isso, é usada uma função que calcula o *hash* do identificador do *peer*, normalmente seu endereço IP, mapeando-o para um inteiro de m bits chamado de *ID de nó*.

$$\text{ID do nó} = \text{hash}(\text{Endereço IP do peer})$$

Uma função de *hash* é uma função matemática que cria uma saída a partir de uma entrada. No entanto, a DHT utiliza alguma das funções de *hash* criptográficas, como os algoritmos da família Algoritmo de *Hash* Seguro (SHA – Secure Hash Algorithm) que são resistentes à colisão, o que significa que a probabilidade de duas entradas serem mapeadas para a mesma saída é muito baixa. Discutiremos esses algoritmos de *hash* no Capítulo 10.

Calculando o *hash* do identificador do objeto

A função de *hash* também é aplicada sobre o nome do objeto (por exemplo, um arquivo) a ser compartilhado, criando um inteiro de m bits no mesmo espaço de endereços. O resultado, no jargão da DHT, é chamado *chave*.

$$\text{chave} = \text{hash}(\text{Nome do objeto})$$

Na DHT, um objeto costuma ser identificado pelo par (*chave*, *valor*) no qual a *chave* é o *hash* do nome do objeto e o *valor* é o objeto ou uma referência ao objeto.

Armazenando o objeto

Existem duas estratégias para armazenar um objeto: o método direto e o indireto. No método direto, o objeto é armazenado no nó cujo ID seja de alguma forma o *mais próximo* à chave no anel. O termo *mais próximo* é definido de forma diferente em cada protocolo. Essa estratégia muito provavelmente envolve a transferência do objeto, que é movido do computador onde originalmente se encontrava. No entanto, a maioria dos sistemas de DHT utiliza o método indireto devido à sua maior eficiência. O *peer* que possui o objeto conserva esse objeto, mas uma referência para ele é criada e armazenada no nó cujo ID é o mais próximo ao ponto correspondente a sua chave. Ou seja, o objeto físico e a referência ao objeto são armazenados em dois locais distintos. Na estratégia direta, criamos um relacionamento entre o ID do nó que armazena o objeto e a chave dele; na indireta, criamos um relacionamento entre a referência (ponteiro) para o objeto e o nó que a armazena. Em ambos os casos, algum relacionamento é necessário para encontrar o objeto quando o nome do objeto é fornecido. Usaremos o método indireto no restante desta seção.

Exemplo 2.15

Embora o valor normal de m seja 160, para efeitos de demonstração, usamos $m = 5$ para facilitar o tratamento dos nossos exemplos. Na Figura 2.48, consideramos que vários *peers* já se juntaram ao grupo. O nó N5, cujo endereço IP é 110.34.56.20, tem um arquivo chamado *Liberdade* que quer compartilhar com seus pares. O nó calcula o *hash* do nome do arquivo, "Liberdade", para obter a chave = 14. Como o nó *mais próximo* à chave 14 é o nó N17, o nó N5 cria uma referência para o nome do arquivo (chave), seu endereço IP, e o número da porta (e possivelmente algumas outras informações sobre o arquivo) e envia tal referência para ser armazenada no nó N17. Em outras palavras, o arquivo é armazenado em N5, a chave do arquivo é C14 (um ponto no anel DHT), mas a referência para o arquivo é armazenada no nó N17. Veremos mais adiante como os outros nós podem primeiramente encontrar N17, extrair a referência e então usá-la para acessar o arquivo *Liberdade*. Nosso exemplo mostra apenas uma chave no anel, mas em uma situação real, há milhões de chaves e nós no anel.

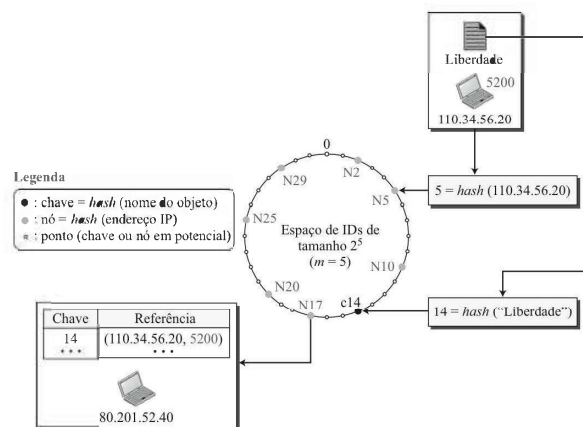


Figura 2.48 Esquema para o Exemplo 2.15.

Roteamento

A função principal da DHT é rotear uma consulta até o nó responsável por armazenar a referência a um objeto. Cada implementação da DHT usa uma estratégia diferente para o roteamento, mas todas seguem a ideia de que cada nó precisa ter um conhecimento parcial sobre o anel para rotear uma consulta para um nó que esteja mais próximo do responsável.

Chegada e saída de nós

Em uma rede P2P, cada *peer* pode ser um *desktop* ou um *laptop*, ligado ou desligado. Quando um *peer* executa o *software* da DHT, ele entra na rede; quando o computador é desligado ou o *peer* fecha o *software*, ele sai da rede. Uma implementação de DHT precisa estabelecer uma estratégia clara e

eficiente para lidar com a chegada e partida dos nós e com o efeito disso sobre o restante dos *peers*. A maioria das implementações de DHT trata a falha de um nó como se ele tivesse saído da rede.

2.4.3 Chord

Existem vários protocolos que implementam sistemas de DHT. Nesta seção, apresentamos três deles: Chord, Pastry e Kademlia. Selecionamos o protocolo Chord por sua simplicidade e sua abordagem elegante para o roteamento de consultas. Em seguida, discutimos o protocolo Pastry porque ele usa uma abordagem diferente do Chord e sua estratégia de roteamento é muito próxima daquela do protocolo Kademlia, que é usado na rede de compartilhamento de arquivos mais popular atualmente, o BitTorrent.

O Chord foi publicado por Stoica *et al.*, em 2001. A seguir, analisaremos brevemente as principais características desse algoritmo.

Espaço de identificadores

Dados e nós no Chord são representados por identificadores de m bits que criam um espaço de identificadores contendo 2^m pontos distribuídos em um círculo no sentido horário. Referimo-nos à identificação de um dado como c (de *chave*) e ao identificador de um *peer* como N (de *nó*). A aritmética no espaço é feita em módulo 2^m , o que significa que os identificadores são cíclicos, indo até $2^m - 1$ e de volta a 0. Embora algumas implementações usem uma função de *hash* resistente a colisões, como o SHA1 com $m = 160$, aqui usamos $m = 5$ para simplificar a discussão. O *peer* mais próximo para o qual $N \geq c$ é chamado de sucessor de c e armazena o valor (c, v) , onde c é a chave (*hash* do objeto de dados) e v é o valor (informações sobre o *peer* servidor que possui o objeto). Ou seja, um objeto de dados, por exemplo, um arquivo, é armazenado no *peer* que possui aquele objeto, mas o valor do *hash* do objeto (chave c) e as informações sobre o *peer* (valor v) são armazenados no sucessor de c na forma do par (c, v) . Isso significa que o *peer* que armazena o objeto de dados e o *peer* que detém o par (c, v) não são necessariamente os mesmos.

Tabela de derivação

Um nó no algoritmo Chord deve ser capaz de resolver uma consulta: dada uma chave, o nó deve ser capaz de encontrar o identificador do nó responsável por aquela chave ou encaminhar a consulta para outro nó. O encaminhamento, contudo, requer que cada nó possua uma tabela de roteamento. O Chord exige que cada nó conheça m nós sucessores e um predecessor. Cada nó cria uma tabela de roteamento, chamada no Chord de tabela de derivação (*finger table*), que se parece com a Tabela 2.14. Note que a chave-alvo na linha i é $N + 2^{i-1}$.

A Figura 2.49 mostra apenas a coluna de sucessores para um anel com alguns nós e chaves. Perceba que a primeira linha ($i = 1$) efetivamente fornece o sucessor do nó. Também adicionamos o ID do nó predecessor, algo necessário, como veremos mais adiante.

Tabela 2.14 Tabela de derivação (<i>finger table</i>).			
i	Chave-alvo	Sucessor da chave-alvo	Informação sobre o sucessor
1	$N + 1$	Sucessor de $N + 1$	Endereço IP e porta do sucessor
2	$N + 2$	Sucessor de $N + 2$	Endereço IP e porta do sucessor
$\vdots$	$\vdots$	$\vdots$	$\vdots$
m	$N + 2^{m-1}$	Sucessor de $N + 2^{m-1}$	Endereço IP e porta do sucessor